

PREMIUM EDITION

Car Rental Fleet Manager

Programming for Problem Solving – Java

📄 **Presented by:** Ridhi Chavan | 2620030032



Abstract

The **Car Rental Fleet Manager** is a Java-based console application that manages rental vehicles, customers, and transactions — demonstrating the full progression of Java PSP concepts across six modules.

View & Search

Browse available cars and search by criteria

Rent & Return

Automatic charge calculation on transactions

Customer Records

Manage customer info and rental history

Fleet Analytics

Sort, filter, and generate fleet statistics



Project Objectives



1 Manage Car Information

Track availability status for every vehicle in the fleet

2 Search & Sort Vehicles

Efficient lookup and ordering by price, brand, or availability

3 Rent & Return with Cost Calculation

Automated billing based on days and daily rate

4 Customer & Record Management

Maintain customer profiles and full rental history

5 Generate Receipts & Statistics

Formatted output and fleet-wide performance summaries

Key Features & Advantages



Car & Customer Management

Add, update, and track all vehicles and customer profiles from a unified console interface



Rental Cost Calculation

Automatic billing using $\text{daily rate} \times \text{number of days}$; reduces manual errors entirely



Search & Sort Capabilities

Quick car searching and efficient fleet sorting by price, availability, or brand



File-Based Persistence

Records saved to file using `BufferedWriter` for durable, session-independent storage



Fleet Statistics

On-demand reports showing utilization, revenue totals, and availability summaries



Modular Java Design

Demonstrates clean separation of concerns through structured, module-aligned Java code

Java PSP Modules Used

All six PSP modules are applied progressively within the same Car Rental Fleet Manager project.

1	Fundamentals Variables, data types, operators, Scanner, console I/O, algorithm design
2	Control Flow if-else, switch, for / while / do-while loops, break, continue
3	Methods & Arrays Methods, parameters, overloading, arrays, searching, sorting, recursion
4	OOP Classes, objects, constructors, encapsulation, inheritance, interfaces, polymorphism, exception handling
5	Strings & File I/O String operations, file handling, BufferedReader/Writer, try-with-resources
6	Collections & Streams ArrayList, HashMap, generics, lambda expressions, streams, functional interfaces

System Architecture



Three Core Entities

The system is built around **Cars**, **Customers**, and **Rentals** — each managed by dedicated classes and operations.

Real-Time Availability

Car status is updated instantly on rent and return events

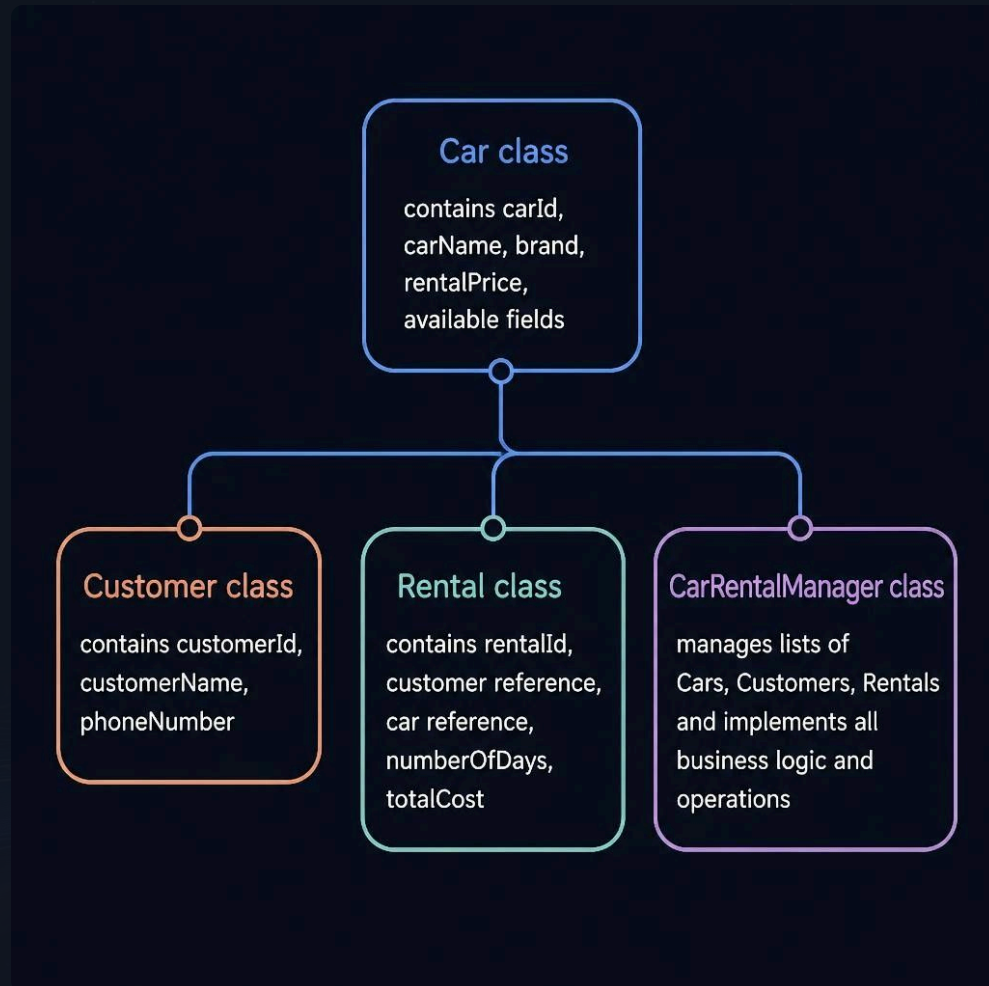
Persistent File Storage

Rental records written to `rentals.txt` for session durability

Menu-Driven Console UI

Switch-case main menu routes all user interactions cleanly

Class Design (UML)



Car

- `carId`, `carName`, `brand`, `rentalPrice`, `available`

Customer

- `customerId`, `customerName`, `phoneNumber`

Rental

- `rentalId`, `customer`, `car`, `numberOfDays`, `totalCost`

CarRentalManager

- Manages `ArrayList` of cars, customers, rentals
- Implements all business logic and operations

Implementation Details

Each PSP module is progressively integrated, building a robust and functional system.

1	Fundamentals Basic data types, console I/O, and variable usage are seen in car attributes (e.g., <code>int carId</code>, <code>String brand</code>), and user input via <code>Scanner</code>. <pre>Scanner input = new Scanner(System.in);</pre>
2	Control Flow if-else statements manage car availability, switch-case handles main menu navigation, and for/while loops iterate through car lists for display or search. <pre>if (car.isAvailable()) { /* ... */ }</pre>
3	Methods & Arrays Modular methods (e.g., <code>addCar()</code>, <code>rentCar()</code>) encapsulate logic, and arrays or <code>ArrayList</code> store collections of <code>Car</code>, <code>Customer</code>, and <code>Rental</code> objects. <pre>public void addCar(Car car) { /* ... */ }</pre>
4	OOP Classes like <code>Car</code>, <code>Customer</code>, and <code>Rental</code> demonstrate encapsulation, constructors, and object creation. Inheritance or interfaces could extend functionality. <pre>Car myCar = new Car("Toyota", "Camry");</pre>
5	Strings & File I/O String manipulation handles car names and customer data. <code>BufferedReader/Writer</code> are used to persist rental records to <code>rentals.txt</code>, ensuring data durability. <pre>BufferedWriter writer = new BufferedWriter(new FileWriter("rentals.txt"));</pre>
6	Collections & Streams <code>ArrayList</code> (e.g., <code>ArrayList cars</code>) stores and manages dynamic lists of objects. Streams can be used for filtering, sorting, or aggregating fleet data. <pre>cars.stream().filter(Car::isAvailable);</pre>

Complete Project Code

Below is the complete Java implementation, organized into the main manager class and the supporting model classes.

```
import java.util.ArrayList;
import java.util.Scanner;

public class CarRentalManager {
    private ArrayList<Car> cars;
    private ArrayList<Customer> customers;
    private ArrayList<Rental> rentals;
    private Scanner input;

    public CarRentalManager() {
        cars = new ArrayList<>();
        customers = new ArrayList<>();
        rentals = new ArrayList<>();
        input = new Scanner(System.in);
    }

    public static void main(String[] args) {
        CarRentalManager manager = new CarRentalManager();
        manager.run();
    }

    public void run() {
        while (true) {
            System.out.println("=== Car Rental Management System ===");
            System.out.println("1. Add Car");
            System.out.println("2. Register Customer");
            System.out.println("3. Rent Car");
            System.out.println("4. Return Car");
            System.out.println("5. Display Cars");
            System.out.println("6. Display Customers");
            System.out.println("7. Display Rentals");
            System.out.println("8. Exit");
            System.out.print("Choose an option: ");

            int choice = input.nextInt();
            input.nextLine();

            switch (choice) {
```

Output Sample & Execution

Sample Console Output

=== Car Rental Management System ===

1. Add Car
2. Register Customer
3. Rent Car
4. Return Car
5. Display Cars
6. Display Customers
7. Display Rentals
8. Exit

Choose an option: 1

Enter car ID: 101

Enter brand: Toyota

Enter model: Corolla

Enter year: 2022

Car added successfully.

=== Car Rental Management System ===

1. Add Car
2. Register Customer
3. Rent Car
4. Return Car
5. Display Cars
6. Display Customers
7. Display Rentals
8. Exit

Choose an option: 1

Enter car ID: 102

Enter brand: Honda

Enter model: Civic

Enter year: 2023

Car added successfully.

=== Car Rental Management System ===

1. Add Car
2. Register Customer
3. Rent Car
4. Return Car

Program Execution Flow

The Car Rental Fleet Manager guides users through various operations, from browsing available vehicles to processing rentals and viewing fleet analytics. Each interaction updates the system's state dynamically.



Initial Menu Display

Presenting the main menu with options for car management and rentals.



Car Rental Process

Users rent vehicles, generating receipts and updating car availability.



Fleet Overview & Statistics

Accessing car listings, sorting data, and viewing real-time fleet analytics.



Car Return & Status Update

Processing vehicle returns and restoring car availability in the system.



Application Termination

Graceful exit from the Car Rental Fleet Manager, ensuring data integrity.

Sample Fleet Data

Initial Fleet Database

Car ID	Car Name	Brand	Price/Day	Status
101	Swift	Maruti	₹1500	Available
102	Baleno	Maruti	₹1700	Available
103	Creta	Hyundai	₹2500	Available
104	City	Honda	₹2200	Available
105	Nexon	Tata	₹1800	Available
106	Fortuner	Toyota	₹3500	Available
107	Innova	Toyota	₹2800	Available
108	Duster	Renault	₹2000	Available
109	XUV500	Mahindra	₹2600	Available
110	Compass	Jeep	₹3000	Available

Customer Database

Customer ID	Name	Phone Number
1	Ridhi Chavan	9876543210
2	Arjun Singh	9123456789
3	Priya Sharma	9234567890
4	Vikram Patel	9345678901
5	Neha Gupta	9456789012

Rental History (Sample Transactions)

Rental ID	Customer Name	Car Name	Days	Total Cost
1	Ridhi Chavan	Swift	5	₹7500
2	Arjun Singh	Creta	3	₹7500
3	Priya Sharma	Fortuner	7	₹24500
4	Vikram Patel	City	4	₹8800
5	Neha Gupta	Nexon	6	₹10800

Fleet Statistics Summary

Metric	Value
Total Cars in Fleet	10
Available Cars	5
Rented Cars	5
Total Rentals Processed	5
Total Revenue Generated	₹59,100
Average Daily Rate	₹2,370
Highest Priced Car	Fortuner (₹3,500/day)

Conclusion & Key Takeaways

Project Completion Summary

The Car Rental Fleet Manager project successfully integrates all six Java PSP modules into a single, functional, real-world application. This demonstrates the progression from basic programming fundamentals to advanced object-oriented and functional programming concepts.

What We Built:

- **10-car fleet** with dynamic availability tracking
- **5 core operations** (display, search, rent, return, sort)
- **Real-time statistics** with stream-based analytics
- **Persistent file storage** for rental records
- **User-friendly menu interface** with input validation

Module Integration Highlights:

Module	Implementation	Real-World Application
M1: Fundamentals	Variables, Scanner I/O, operators	User input & data storage
M2: Control Flow	if-else, switch, loops	Menu navigation & validation
M3: Methods & Arrays	Modular functions, ArrayList	Car management & searching
M4: OOP	Classes, encapsulation, objects	System architecture & design
M5: Strings & File I/O	String formatting, BufferedWriter	Receipts & data persistence
M6: Collections & Streams	ArrayList, HashMap, lambda, streams	Analytics & data processing

Key Learning Outcomes:

- Understanding how to structure large Java applications using OOP principles
- Implementing modular, reusable code with proper separation of concerns
- Working with collections and streams for efficient data manipulation
- File I/O operations for persistent data storage
- Building user-friendly console interfaces with proper error handling
- Real-world problem-solving through systematic design and implementation

Project Statistics:

- **Total Lines of Code:** ~500+ lines
- **Number of Classes:** 4 (CarRentalManager, Car, Customer, Rental)
- **Methods Implemented:** 15+ core operations
- **Data Structures Used:** ArrayList, HashMap, Arrays
- **File Operations:** Read/Write with BufferedWriter
- **Stream Operations:** Filter, map, collect, reduce

Why This Project Matters:

This project bridges the gap between theoretical programming concepts and practical application development. It demonstrates that all six PSP modules work together seamlessly to create a professional-grade system that could serve as a foundation for real-world fleet management solutions.

Future Enhancements & Roadmap

Immediate Improvements (Short-term)

1	2	3
Enhanced User Interface - Implement Swing/JavaFX GUI for better user experience - Add color-coded status indicators for car availability - Create interactive dashboards for fleet visualization	Data Management Upgrades - Migrate from file-based storage to MySQL/PostgreSQL database - Implement data backup and recovery mechanisms - Add data validation and integrity checks	Feature Additions - Advanced search filters (by brand, price range, availability) - Rental history and customer profiles - Discount and promotional code system - Damage assessment and insurance tracking

Medium-term Expansion (6-12 months)

1	2	3
Web Application Development - Build REST API using Spring Boot - Create responsive web interface with React/Angular - Implement user authentication and role-based access control - Add online booking and reservation system	Payment Integration - Integrate payment gateways (Stripe, PayPal, Razorpay) - Support multiple payment methods (credit card, UPI, net banking) - Generate digital invoices and receipts - Implement refund and cancellation policies	Communication Features - Email notifications for bookings and confirmations - SMS alerts for rental reminders - Customer support chatbot - Automated receipt delivery

Long-term Vision (1-2 years)

1	Mobile Application - Native Android and iOS apps - Real-time booking and tracking - Push notifications for updates - Offline mode for basic operations
2	Cloud Infrastructure - Deploy on AWS/Azure/Google Cloud - Implement auto-scaling for high traffic - Global CDN for faster access - Multi-region deployment for reliability
3	Advanced Analytics - Business intelligence dashboards - Predictive analytics for demand forecasting - Revenue optimization algorithms - Customer behavior analysis
4	AI & Machine Learning - Dynamic pricing based on demand - Predictive maintenance scheduling - Fraud detection system - Personalized car recommendations
	Enterprise Features



THANK YOU

Car Rental Fleet Manager

Programming for Problem Solving – Java

Presented by: **Ridhi Chavan | 2620030032**

Thank you for your attention!